ASSIGNMENT 2 – CO2

Functions and Object-Oriented Concepts in C++

Assignment Report – Q1 to Q7 (K5 – Evaluate)

Course	Data Structures and Algorithms (DSA01)
Student name	A.MOHAMMED KALIFA
Register number	192411078
Nature	Individual – 50 marks
Submission date	31/10/2026
Compiler	g++ 13.3.0 (Ubuntu 13.3.0-6ubuntu2~24.04.1), -std=c++17 -Wall -Wextra
GitHub repository	https://github.com/mohammedkalifa7/DSA0103C-.git

CO2 coverage: function prototyping, call by reference, return by reference, inline functions, default arguments, function overloading, classes, member functions, access specifiers, arrays within a class, static data members and member functions, arrays of objects and friend functions.

Contents

Overview and results at a glance

Q1 – Call by Value, by Reference and by Pointer (6 marks)

Q2 – Return by Reference and Its Dangers (4 marks)

Q3 – Default Arguments and Function Overloading (6 marks)

Q4 – Specifying a Class with an Array Inside It (6 marks)

Q5 – Static Members and Arrays of Objects (7 marks)

Q6 – Friend Functions and Inline Functions (8 marks)

Q7 – Diagnose, Redesign and Validate (13 marks)

Appendix A – README.md

Appendix B – GitHub submission checklist and commit commands

Appendix C – Summary of complexity results

Overview and results at a glance

This report answers every task of Assignment 2. For each question the C++ programs were compiled with g++ 13.3.0 and executed; the outputs, compiler messages and timings shown are the real ones produced by those runs (terminal captures are included as figures and as files in each `QN/screenshots/` folder). Predictions are recorded next to the observed results, and each question ends with the alternative-concept and complexity analysis requested.

Q	Topic	Marks	Key result
1	Call by value / reference / pointer	6	Fee 45000 → unchanged by value, 40500 by reference, 36450 by pointer; chose "return the new fee" because stacked concessions depend on order (25819 vs 25820)
2	Return by reference	4	`highest(...) += 2` works; returning a local reference → warning + segfault; vector growth → heap-use-after-free
3	Defaults and overloading	6	Ambiguity of `charge(2.5)` explained and removed; six predicted selections all correct
4	Class with an array	6	Private circular/shifting history; shifting cost grows linearly with capacity (7.7 → 128.5 ns for k = 10 → 1000)
5	Static members, arrays of objects	7	`101 103 104 4`; BusTicket expected output reproduced; count survives object destruction
6	Friend and inline functions	8	`205 false`; 7/7 tests; inline ≈ 1.1–1.45× faster at -O2, no effect at -O0
7	Diagnose, redesign, validate	13	Circular buffer: 43× faster deposits at k = 1000, identical output, but 1.8–3.5× slower reads

Method

- Predictions were written from a hand trace of the code; where the paper asks for "before running" the prediction is shown first and compared with the observed output.
- Timings use `std::chrono::steady_clock`, medians of 7 repetitions, results consumed through a `volatile` sink so the optimiser cannot delete the work. They were taken on a

shared 2-core cloud machine, so absolute numbers vary by about ±10 % between runs; ratios and trends are what matter.

- Compiler diagnostics (ambiguity, private access, static-function misuse, dangling reference) were obtained by compiling deliberately faulty files that are kept in the repository next to the working versions.

Q1 – Call by Value, by Reference and by Pointer (6 marks)

1.1 Predict the output

Program from the question (`Q1/predict.cpp`). Prediction written by tracing the code by hand before running:

Statement	Reasoning	Predicted
`change(a, b, &c)`	a is a copy (stays 1); b is an alias of the caller's b (2 → 12); c is changed through its address (3 → 13).	`1 12 13`
`bigger(a, c) = 100;`	`bigger` returns an `int&` to the larger of a (1) and c (13), i.e. to c. The call is an lvalue, so 100 is stored into c; a is untouched.	`1 100`
`show(1), show(1,2), show(1,2,3)`	1+5+10 = 16 (y and z defaulted); 1+2+10 = 13 (only z defaulted); 1+2+3 = 6 (no default used). Each value is followed by a space, then one newline.	`16 13 6` (trailing space)

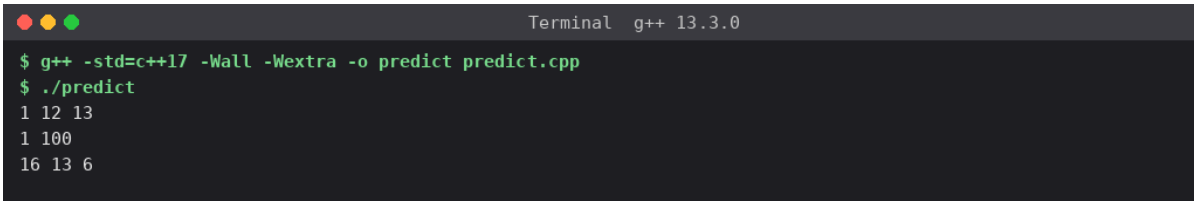


Figure 1.1 – Actual output of predict.cpp

Prediction vs actual: identical – no differences. The only subtle point is the trailing space after `6`, which comes from `' '` being printed after every value inside `show()`.

Which variables changed and why. In `change()` the first call changed `b` and `c` but not `a`: `a` is passed by value (a private copy on the callee's stack), `b` by reference (the parameter *is* the caller's `b`) and `c` by pointer (the callee dereferences the address of `c`). In `bigger(a, c) = 100`, only `c` changed because the function returned a reference to the larger argument. Default arguments: `show(1)` used **both** defaults (`y = 5`, `z = 10`); `show(1, 2)` used only `z = 10`; `show(1, 2, 3)` used **none**.

1.2 Complete the code

TODO 1 completed so the caller's fee changes (`Q1/complete.cpp`; the rest of the program is unchanged):

```
void applyConcession(long long& fee, int percent)
{
    // TODO 1 (completed): reduce the caller's fee by percent (integer arithmetic)
    fee -= fee * percent / 100;
}
```

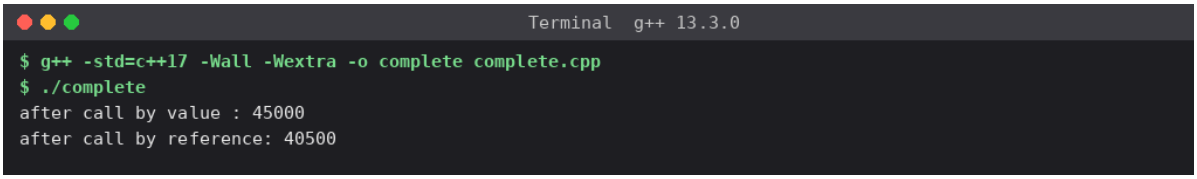


Figure 1.2 – Output matches the expected output (45000, 40500)

Why call by value leaves the fee at 45000. `applyByValue(hostelFee, 10)` copies the value 45000 into the parameter `fee`, a separate `long long` object in the callee's stack frame. `fee -= fee * percent / 100` changes 45000 → 40500 in that copy only; the copy is destroyed when the

function returns, and `hostelFee` in `main` was never touched. With integer arithmetic 45000 × 10 / 100 = 4500, so by reference 45000 − 4500 = 40500.

1.3 The concession function three ways

`Q1/three_ways.cpp` implements `byValue(long long, int)`, `byReference(long long&, int)` and `byPointer(long long*, int)` (plus a return-value version and a `const&` preview). Predictions were made before running; each call applies 10 % to the current stored fee.

Call	Predicted stored fee	Observed	Match
`byValue(fee, 10)`	45000 (inside: 40500)	45000 (inside: 40500)	Yes
`byReference(fee, 10)`	40500	40500	Yes
`byPointer(&fee, 10)`	36450 (40500 − 4050)	36450	Yes

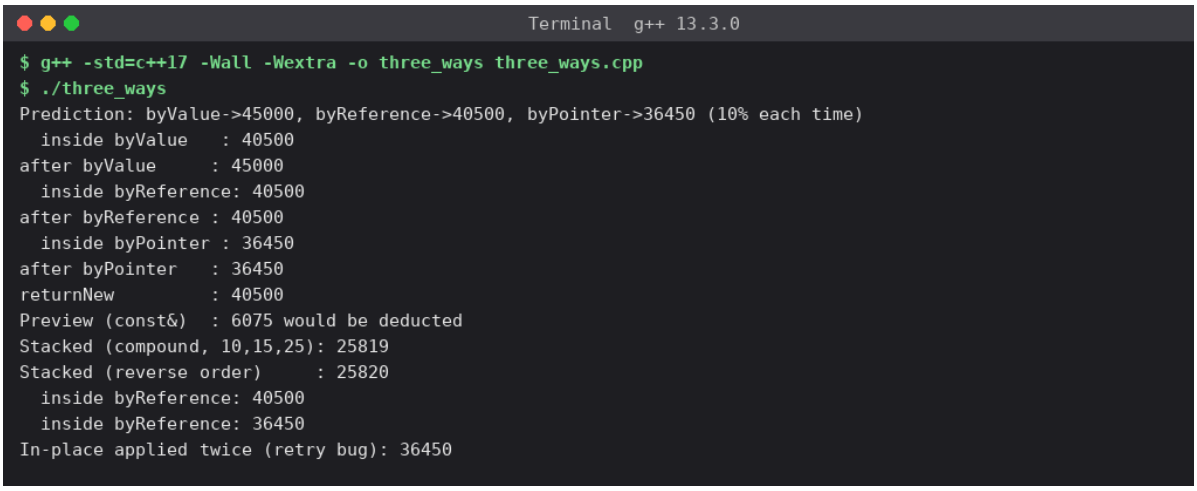


Figure 1.3 – three_ways.cpp output

```
void byValue(long long fee, int percent)         { fee -= fee * percent / 100; }
void byReference(long long& fee, int percent)    { fee -= fee * percent / 100; }
void byPointer(long long* fee, int percent)      { if (!fee) return; *fee -= *fee *
percent / 100; }
long long returnNew(long long fee, int percent) { return fee - fee * percent / 100; }
// full source with printing: Q1/three_ways.cpp
```

Inquiry – where each version stores its parameter

Version	Where the parameter lives	Can it change caller's fee?
By value	A **new** `long long` (8 bytes) in the callee's stack frame holding a copy of the value.	No – it only modifies its own copy.
By reference	No new object: `fee` is another name (alias) for the caller's variable (the compiler typically passes a hidden address, but semantically nothing is copied).	Yes – every use of `fee` is the caller's object.
By pointer	A new pointer object (8 bytes) in the callee's frame that holds the **address** of the caller's variable; the fee itself stays in the caller.	Yes – through `*fee`. (The pointer itself is a copy, so reassigning it would not affect the caller.)

Only two versions change the caller's fee because only they give the function access to the caller's object; by value gives it a copy. A **`const` reference** (`const long long&`, or `const Student&`)

should be used when the function only **reads** the argument and copying would be wasteful – large objects such as a `Student` with strings/arrays. It avoids the copy and the compiler rejects any accidental modification. For small types such as `long long` plain by-value is just as cheap and simpler.

Design decision – in place or return the new fee?

Several concessions may apply to one student (merit 10 %, sports 15 %, single parent 25 %). I measured what happens when they are stacked on 45000 with integer arithmetic:

Scenario	Result
10 % → 15 % → 25 % applied one after another	25819
25 % → 15 % → 10 % (reverse order)	25820
In-place `byReference` called twice by a retry/bug	36450 (discount applied twice)

> **Decision: return the new fee** (a pure function `long long applyConcession(long long fee, int percent)`), and let the caller write `fee = applyConcession(fee, p)`. Reasons: (1) the result of stacking concessions depends on the **order** (25819 vs 25820 because of integer truncation) – a pure function makes the order explicit in the calling code or in one `finalFee(fee, list)` function; (2) it is composable (`applyConcession(applyConcession(f, 10), 15)`) and trivially testable; (3) it cannot silently double-apply when a call is retried – the in-place version did; (4) the original fee is still available for receipts and audits. An in-place (reference) version remains reasonable only for one well-defined, one-time update of a very large record.

Alternative concept and complexity analysis

Return-value solution: `long long applyConcession(long long fee, int percent) { return fee - fee * percent / 100; }` and in `main`: `hostelFee = applyConcession(hostelFee, 10);` – prints 40500 (verified in `three_ways.cpp`, line "returnNew : 40500").

Passing style	Time	Space copied per call	Large `Student` object (s bytes)
By value	O(1)	8 bytes (`long long`)	copies all s bytes – O(s) time and space per call
By reference	O(1)	0 bytes copied (alias, one address)	0 bytes copied
By pointer	O(1)	8 bytes (the pointer)	8 bytes
By `const&`	O(1)	0 bytes copied (read-only)	0 bytes copied
Return new fee	O(1)	8 bytes returned (usually in a register)	returned object copied/moved: O(s) unless elided

All versions do one multiplication, one division and one subtraction, so time is O(1) for the `long long`. The difference appears only in the **space and copy cost** for large objects, where by-value/return-by-value cost grows with the object size while references and pointers stay at one address.

Evidence delivered: three implementations, predictions before execution, observed results and a defended interface design (above).

Q2 – Return by Reference and Its Dangers (4 marks)

2.1 Implementation

`Q2/marks.cpp` contains `int& highest(int[], int)` (reference to the best mark), `const int& highestRO(...)`, `int highestCopy(...)` and the deliberately wrong `int& badReference()`.

```
// Q2 - Returning references: highest mark, const ref, copy, and the dangling-reference bug
#include <iostream>

int& highest(int marks[], int n)              // reference to the array element itself
{
    int best = 0;
    for (int i = 1; i < n; ++i)
        if (marks[i] > marks[best]) best = i;
    return marks[best];                       // safe: element outlives the call
}
const int& highestRO(const int marks[], int n) // read-only view of the element
{
    int best = 0;
    for (int i = 1; i < n; ++i)
        if (marks[i] > marks[best]) best = i;
    return marks[best];
}
int highestCopy(const int marks[], int n)     // returns a copy of the value
{
    int best = marks[0];
    for (int i = 1; i < n; ++i)
        if (marks[i] > best) best = marks[i];
    return best;
}
int& badReference()                           // DANGER: returns reference to a local
{
    int local = 99;
    return local;                             // warning: reference to local variable
returned
}
int main(int argc, char**)
{
    int marks[] = {56, 71, 88, 64, 79};
    const int n = 5;

    highest(marks, n) += 2;                   // function call on the LEFT of an assignment
    std::cout << "after highest(marks) += 2 : " << marks[2] << '\n';
    highest(marks, n) = 90;
    std::cout << "after highest(marks) = 90 : " << marks[2] << '\n';

    int  c = highestCopy(marks, n); c += 5;   // copy: array unaffected
    std::cout << "copy changed to " << c << ", array still " << marks[2] << '\n';

    const int& r = highestRO(marks, n);       // r = alias, cannot modify through r
    // r += 1;   // error: assignment of read-only reference
    marks[2] = 91;
    std::cout << "const ref sees later change: " << r << '\n';

    std::cout << "address of marks[2] = " << &marks[2] << ", &highest() = " <<
&highest(marks, n) << '\n';

    if (argc > 1) {                           // run with any argument to trigger the UB
        int& d = badReference();              // undefined behaviour from here on
        std::cout << "dangling reference d (UB): " << d << std::endl;
    } else std::cout << "(run with an argument to execute the dangling-reference call)\n";
}
```

2.2 What the program shows

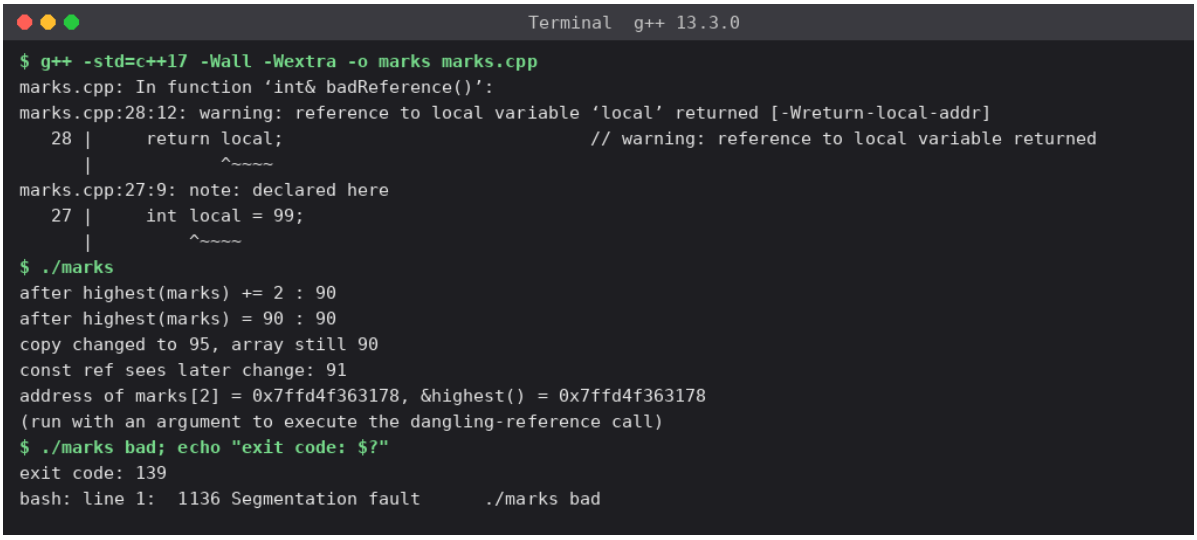


Figure 2.1 – Compiler warning, normal run, and the dangling call (segmentation fault, exit code 139)

Observation	Evidence	Explanation
Caller changes the array element	`highest(marks, n) += 2` gave 90 (88+2); `highest(marks, n) = 90` kept 90	The function returns an **lvalue** referring to `marks[2]` itself, so the call can sit on the left of = or +=. Same address printed for `&marks[2]` and `&highest(...)`.
Compiler warning for local reference	`warning: reference to local variable 'local' returned [-Wreturn-local-addr]`	`local` lives in the function's stack frame, which is released at `return`. The reference would refer to an object whose lifetime has ended.
Why it is undefined behaviour	`./marks bad` → Segmentation fault, exit 139 (also at -O2 and under AddressSanitizer: SEGV on address 0x0)	The standard gives no guarantee: the slot may be reused, hold garbage, or (as GCC does here) be replaced by a null reference, which crashes. Never rely on the "it printed 99" outcome seen with other compilers.
Reference vs const reference vs copy	copy changed to 95 while array stayed 90; the const reference saw a later change to 91	See table below.

Return type	Alias of the array element?	Caller can modify element?	Risk
`int&`	Yes	Yes (`f() += 2`, `f() = 90`)	Dangling if the element/ object dies or moves
`const int&`	Yes	No – read-only view (`r += 1` is a compile error); still sees later changes (printed 91)	Same dangling risk, but no accidental writes
`int` (copy)	No – an independent value	No – changing the copy (95) left the array at 90	None; costs one copy and cannot be used for moderation

2.3 Changed condition – the register becomes a growing std::vector

`Q2/vector_grow.cpp` obtains `int& top = highest(marks)` from a vector with capacity 3, then calls `push_back`. When the size exceeds the capacity the vector allocates a bigger block, moves the elements and frees the old block, so `top` still refers to freed memory.

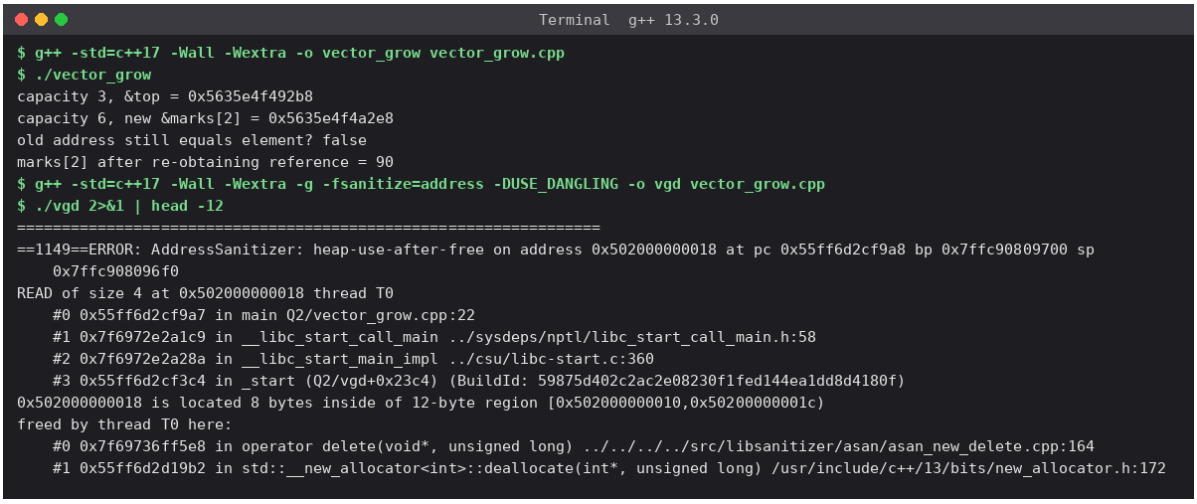


Figure 2.2 – Capacity 3 → 6, element address changes; AddressSanitizer reports heap-use-after-free

Explanation. A reference (like a pointer or iterator) into a `std::vector` is tied to the **address** of the element. Growth beyond `capacity()` reallocates: capacity went from 3 to 6 and the element moved from …`42b8` to …`52e8`. The old reference still holds the old address, which has been freed, so any read/write through it is undefined behaviour (ASan: "heap-use-after-free … 8 bytes inside of 12-byte region … freed by thread T0"). **Justified response:** never keep a reference across an operation that can grow the container – re-obtain it after the `push_back` (shown: 88 → 90 correctly), reserve enough capacity in advance, or hold an **index** instead of a reference.

2.4 Alternative concept and complexity analysis

If the highest mark is returned as an index or a pointer (`Q2/alternatives.cpp`) the calling code becomes:

```
marks[highestIndex(marks)] += 2;   // index  : call site must index the container again
*highestPtr(marks)         += 2;   // pointer: dereference with *, may be nullptr
highestRef(marks)          += 2;   // reference: cleanest, cannot be null
```

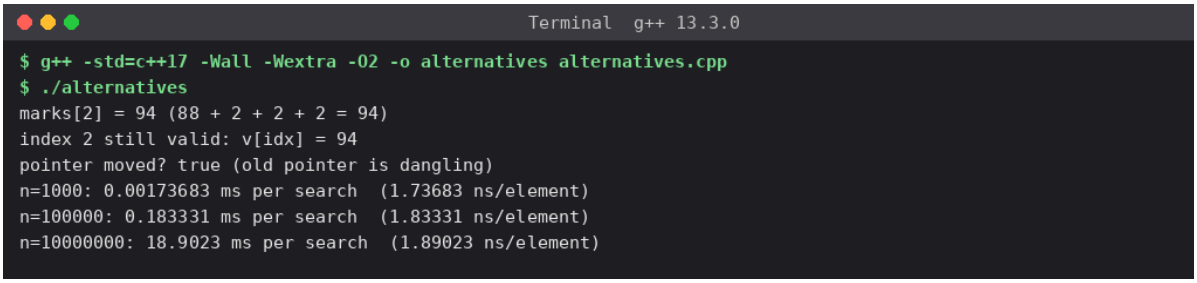


Figure 2.3 – All three call styles give 94; timing of the O(n) search

Version	Time to find highest	Extra space	Safety if array later becomes a growing vector
Reference `int&`	O(n) – one pass	O(1)	Dangles after reallocation; cannot signal "empty"
Pointer `int*`	O(n)	O(1) (one pointer)	Dangles after reallocation (observed: "pointer moved? true"); can return nullptr for empty

Version	Time to find highest	Extra space	Safety if array later becomes a growing vector
Index `int`	O(n)	O(1)	**Stays valid** after growth (index 2 still gave 94); only wrong if elements are erased/reordered

Measured search time grows linearly: 1.74 ns/element at n = 1,000, 1.83 at n = 100,000 and 1.89 at n = 10,000,000 (0.0017 ms, 0.18 ms, 18.9 ms) – consistent with O(n). The index is the safest design for a growing vector; the reference is the most convenient for a fixed array.

Q3 – Default Arguments and Function Overloading (6 marks)

3.1 The compiler error (ambiguous call)

```
// Q3 - the broken original: three overloads, one with defaults
#include <iostream>
double charge(double weight)                                          { return 30 + 20 *
weight; }
double charge(double weight, bool express)                            { return
charge(weight) * (express ? 1.5 : 1.0); }
double charge(double weight, bool express = false, bool insured = false) { return
charge(weight, express) + (insured ? 25 : 0); }

int main()
{
    std::cout << charge(2.5) << '\n';   // which one? (double) or (double, bool=false,
bool=false)
}
```

```
Terminal  g++ 13.3.0
$ g++ -std=c++17 -Wall -Wextra -o ambiguous ambiguous.cpp 2>&1 | grep -E "error|candidate"
ambiguous.cpp:5:89: error: call of overloaded ‘charge(double&, bool&)’ is ambiguous
ambiguous.cpp:4:8: note: candidate: ‘double charge(double, bool)’
ambiguous.cpp:5:8: note: candidate: ‘double charge(double, bool, bool)’
ambiguous.cpp:9:24: error: call of overloaded ‘charge(double)’ is ambiguous
ambiguous.cpp:3:8: note: candidate: ‘double charge(double)’
ambiguous.cpp:5:8: note: candidate: ‘double charge(double, bool, bool)’
```

Figure 3.1 – g++ reports "call of overloaded charge(double) is ambiguous"

Explanation. For `charge(2.5)` two functions are viable: `charge(double)` and `charge(double, bool = false, bool = false)` (the omitted arguments are supplied by the defaults). Both take the `double` as an **exact match** and need no conversion for the missing arguments, so neither is better and the call is ambiguous. The same happens inside the third function for `charge(weight, express)`, which matches both `charge(double, bool)` and `charge(double, bool, bool)` – the compiler reports two errors. The overloads and defaults are redundant: the default arguments already create the one- and two-argument forms.

3.2 Redesign

Rules followed: (1) default arguments live in **one** function per kind of input, so every short call is covered once; (2) overloads differ by the **kind** of input – `int` grams, `double` kilograms, `Parcel` object; (3) the grams and Parcel overloads delegate to the kg function, so the pricing rule exists in one place (₹30 base + ₹20/kg, express +50 %, insurance +₹25).

```
// Q3 - Courier charge calculator: defaults + overloading, no ambiguity
#include <iostream>
#include <iomanip>

struct Parcel { double kg; double volumeFactor; };   // volumeFactor: 1.0 normal, >1 bulky

const double BASE = 30.0, PER_KG = 20.0, EXPRESS_PCT = 50.0, INSURANCE = 25.0;

// core calculation - the ONE place with default arguments (rightmost parameters)
double charge(double kg, bool express = false, bool insured = false)
{
    double c = BASE + PER_KG * kg;
    if (express) c += c * EXPRESS_PCT / 100;
    if (insured) c += INSURANCE;
    return c;
}
// grams as int: converts and delegates (different kind of input -> overload)
double charge(int grams, bool express = false, bool insured = false)
{
    return charge(grams / 1000.0, express, insured);
}
```

```
// Parcel object
double charge(const Parcel& p, bool express = false, bool insured = false)
{
    return charge(p.kg * p.volumeFactor, express, insured);
}

int main()
{
    std::cout << std::fixed << std::setprecision(2);
    std::cout << "1. charge(2)            -> int grams      : " << charge(2) << '\n';
    std::cout << "2. charge(2.0f)         -> float->double  : " << charge(2.0f) << '\n';
    std::cout << "3. charge(2.5)          -> double kg      : " << charge(2.5) << '\n';
    std::cout << "4. charge(2.5, true)    -> kg + express   : " << charge(2.5, true) <<
'\n';
    std::cout << "5. charge(1500, true, true) -> grams,all  : " << charge(1500, true, true)
<< '\n';
    std::cout << "6. charge(Parcel{2.0,1.5})-> Parcel       : " << charge(Parcel{2.0, 1.5})
<< '\n';
}
```

3.3 Predict which function is selected, then run

#	Call	Candidates and ranking	Predicted selection and value
1	`charge(2)`	`int` exact match; `double` needs int→double conversion	`charge(int grams,…)` → 2 g → 30.04
2	`charge(2.0f)`	`double`: float→double is a **promotion**; `int`: float→int is a **conversion**; Parcel: not viable. Promotion beats conversion	`charge(double kg,…)` → 70.00
3	`charge(2.5)`	`double` exact	`charge(double kg,…)` → 80.00
4	`charge(2.5, true)`	`double` exact, `true` fills `express`	`charge(double kg,…)` → 120.00
5	`charge(1500, true, true)`	`int` exact	`charge(int grams,…)` → 1.5 kg: 60 → ×1.5 = 90 → +25 = 115.00
6	`charge(Parcel{ 2.0, 1.5})`	only the `Parcel` overload is viable	`charge(const Parcel&,…)` → 3 kg: 30+60 = 90.00

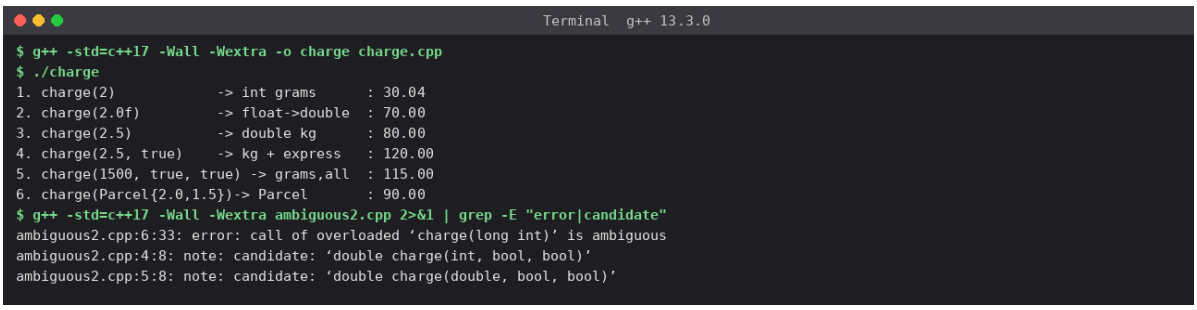


Figure 3.2 – Observed selections: all six match the predictions; no call is ambiguous (compiled with -Wall -Wextra, no warnings)

Prediction vs observation: all six values matched. Note the semantic trap in call 1: `charge(2)` is **2 grams** (30.04), not 2 kg – a consequence of choosing an `int` overload for grams. This is documented in the code and is exactly why `charge(2.0f)` (70.00) behaves differently.

A remaining ambiguity: `charge(2L)` (a `long`) is ambiguous in this design because `long→int` and `long→double` are both *conversions* of equal rank (`Q3/ambiguous2.cpp`, error shown at the

bottom of Figure 3.2). The fix is an explicit cast (`charge(int(2L))`) or a `long` overload; the six designed calls contain no such case.

Inquiry – how the compiler chooses among overloaded functions

- **Collect** all functions with that name visible at the call; **drop** the non-viable ones (wrong number of arguments after counting defaults, or no conversion possible).
- **Rank** each argument conversion: *exact match* (identity, array-to-pointer, adding `const`) > *promotion* (float→double, short/char→int) > *standard conversion* (int→double, double→int, long→int) > *user-defined conversion*.
- The best function must be at least as good on every argument and strictly better on one. If no function wins, the call is **ambiguous** and compilation fails; the return type is never considered.
- **Default arguments must be given from the right** because at the call site arguments are matched left-to-right and only trailing ones can be omitted. If `f(int a = 1, int b)` were legal, `f(5)` could not say whether 5 is `a` or `b`; the compiler therefore requires that once a parameter has a default, all parameters to its right have defaults too.

3.4 Alternative concept and complexity analysis

A single `Parcel` structure with optional (default-initialised) fields plus a function template for raw weights (`Q3/alt_parcel.cpp`) gives the same results for the same inputs (30.04, 80.00, 120.00, and 175.00 for a bulky, fragile, express, insured Parcel).

```
// Q3 alternative: one Parcel struct with optional fields + a function template for raw weights
#include <iostream>
#include <iomanip>
#include <type_traits>

struct Parcel {
    double kg = 0;
    bool   express = false;
    bool   insured = false;
    double volumeFactor = 1.0;      // a NEW option costs one new field, no new overloads
    bool   fragile = false;         // added later: existing calls unaffected
};
double charge(const Parcel& p)
{
    double c = 30 + 20 * p.kg * p.volumeFactor;
    if (p.express) c *= 1.5;
    if (p.insured) c += 25;
    if (p.fragile) c += 15;
    return c;
}
template <class W, class = std::enable_if_t<std::is_arithmetic_v<W>>>
double charge(W weight, bool express = false, bool insured = false)
{   // grams if integral, kg if floating
    double kg = std::is_integral_v<W> ? weight / 1000.0 : double(weight);
    return charge(Parcel{kg, express, insured});
}
int main()
{
    std::cout << std::fixed << std::setprecision(2);
    std::cout << charge(2) << ' ' << charge(2.5f) << ' ' << charge(2.5, true) << ' ' <<
charge(Parcel{2, true, true, 1.5, true}) << '\n';
}
```

Aspect	Overloads + default arguments	Parcel struct / template
Overload resolution time	Done at **compile time**; zero run-time cost	Also compile time; a template adds instantiation time at compile time only
Run-time time	O(1)	O(1)

Aspect	Overloads + default arguments	Parcel struct / template
Space copied per call	`double` + 2 `bool` ≈ 16 bytes (or a const-ref Parcel)	A `Parcel` by const-ref copies nothing; by value copies ~24 bytes
Adding a new option (e.g. fragile)	New parameter in every overload or new overloads → risk of new ambiguities and long positional argument lists (`charge(2.5, false, true, true)`)	One new field with a default; existing calls unchanged; call sites can name the field
Readability of short calls	Best: `charge(2.5, true)`	Needs a Parcel to be built or the template wrapper

Conclusion: for three fixed options, overloads with defaults give the shortest calls; once options keep growing, the Parcel structure scales better because a new option does not change any function signature.

Q4 – Specifying a Class with an Array Inside It (6 marks)

4.1 Class specification

`Q4/account.cpp`: `SavingsAccount` keeps the account number, balance, a private array `last[CAP]` of `Transaction` records (CAP = 10) and a `count`. `getBalance()`/`getAccNo()` are defined **inside** the class (implicitly inline); `deposit()`, `withdraw()`, `miniStatement()` and the private helper `record()` are defined **outside** with the scope resolution operator `SavingsAccount::`.

```
// Q4 - SavingsAccount with a private array of the last N transactions (shifting design)
#include <iostream>
#include <string>
#include <iomanip>

struct Transaction { char type; double amount; double balanceAfter; };   // 'D' or 'W'

class SavingsAccount {
    static const int CAP = 10;          // one constant: change to 50 for the new requirement
    long          accNo;
    double        balance;
    Transaction   last[CAP];            // PRIVATE: only member functions keep count/order
consistent
    int           count;                // number of valid entries (0..CAP)

    void record(char t, double amt);    // declared inside, defined outside
public:
    SavingsAccount(long no, double opening) : accNo(no), balance(opening), count(0) {}

    // defined INSIDE the class (implicitly inline)
    double getBalance() const { return balance; }
    long   getAccNo()   const { return accNo; }

    // defined OUTSIDE using the scope resolution operator
    void deposit(double amt);
    bool withdraw(double amt);
    void miniStatement() const;
};

void SavingsAccount::record(char t, double amt)
{
    if (count == CAP) {                       // 11th transaction: oldest is discarded
        for (int i = 1; i < CAP; ++i)         // shift everything left: O(CAP)
            last[i - 1] = last[i];
        --count;
    }
    last[count++] = Transaction{t, amt, balance};
}
void SavingsAccount::deposit(double amt)
{
    if (amt <= 0) return;
    balance += amt;
    record('D', amt);
}
bool SavingsAccount::withdraw(double amt)
{
    if (amt <= 0 || amt > balance) return false;
    balance -= amt;
    record('W', amt);
    return true;
}
void SavingsAccount::miniStatement() const
{
    std::cout << "Mini-statement for A/C " << accNo << " (last " << count << ")\n";
    for (int i = count - 1; i >= 0; --i)      // newest first
        std::cout << "  " << (last[i].type == 'D' ? "Deposit    " : "Withdrawal")
                  << std::setw(9) << last[i].amount << "  bal " << std::setw(9) <<
last[i].balanceAfter << '\n';
```

```
    std::cout << "Balance: " << balance << '\n';
}

int main()
{
    SavingsAccount a(1001, 1000);
    for (int i = 1; i <= 12; ++i) {           // 12 transactions: first two must fall off
        if (i % 3 == 0) a.withdraw(50 * i); else a.deposit(100 * i);
    }
    a.miniStatement();
    // a.last[0] = Transaction{'D', 1e9, 1e9};   // error: 'last' is private
}
```

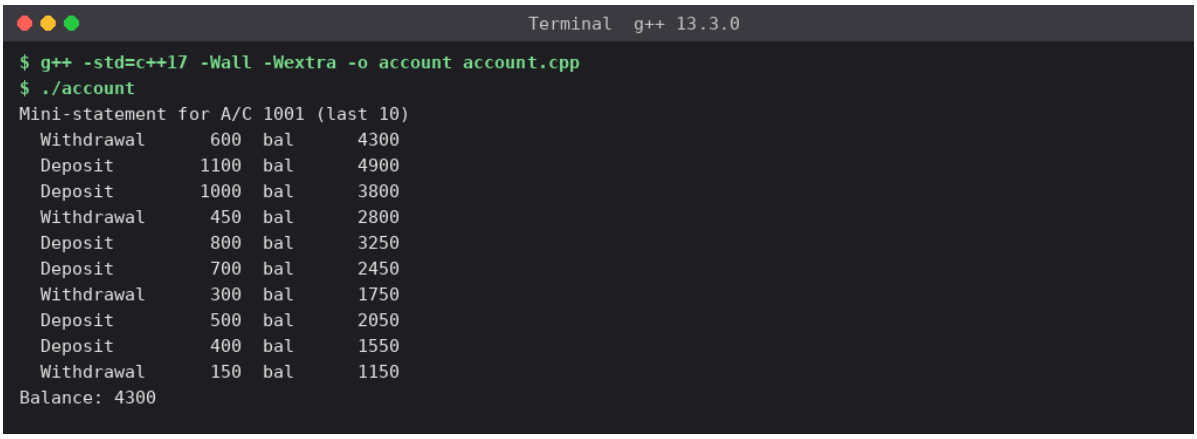


Figure 4.1 – 12 transactions are made; the mini-statement shows only the last 10 (newest first)

Hand check: transactions 1–12 are deposits of 100·i except i = 3, 6, 9, 12 which are withdrawals of 50·i. After transaction 2 the balance is 1300; transaction 3 (withdrawal 150) gives 1150 – this is the oldest line shown, so transactions 1 and 2 were discarded. The newest (12th, withdrawal 600) leaves 4300 = the printed balance.

4.2 Inquiry

Why the array must be private

The array, the `count` and the balance form one **invariant**: `count ≤ CAP`, entries are in time order, and each entry's balance agrees with the account balance. If `last[]` were public any code could write `a.last[0] = …`, overwrite history without changing the balance, or set `count` to 99 and make `miniStatement()` read outside the array. Making it private forces every change through `deposit()`/`withdraw()`, which keep the invariant. Evidence: uncommenting `a.last[0] = …` in `main` gives `error: 'Transaction SavingsAccount::last [10]' is private within this context`. Privacy also lets me change the storage strategy (Section 4.4, Q7) without touching any caller.

What happens when the 11th transaction arrives

Strategy	What happens	Cost of one add when full
Shift the array	Entries 1…9 are copied one slot to the left (overwriting the oldest at index 0), `count` is decremented, and the new transaction is stored at index 9.	CAP − 1 element moves → **O(k)**, k = CAP
Circular index	A `head` index marks the oldest entry. The new transaction overwrites the slot at `head`, then `head = (head + 1) % CAP`. No element moves; reading starts at `head` and wraps around.	One write + one increment → **O(1)**

4.3 Design decision – last 50 transactions

> **Decision: change only the constant `CAP` from 10 to 50 and use the circular-index recording.** Because the capacity appears in exactly one constant, the class interface (`deposit`, `withdraw`, `miniStatement`) and all callers do not change. With the shifting design the same one-line change works too, but the cost per transaction grows 5× (see measurements); with the circular index the per-transaction cost stays the same. A `std::deque` would also work but changes the data member type and allocates dynamically. I verified that both implementations give byte-identical statements at CAP = 50 (Q7).

4.4 Alternative concept and complexity analysis

Alternatives to shifting a fixed array: (a) a `std::deque` with `push_back` and `pop_front` when the size reaches the limit; (b) a circular buffer with a start index. `Q4/bench.cpp` measures the median of 7 runs of 2,000,000 additions with a full buffer.

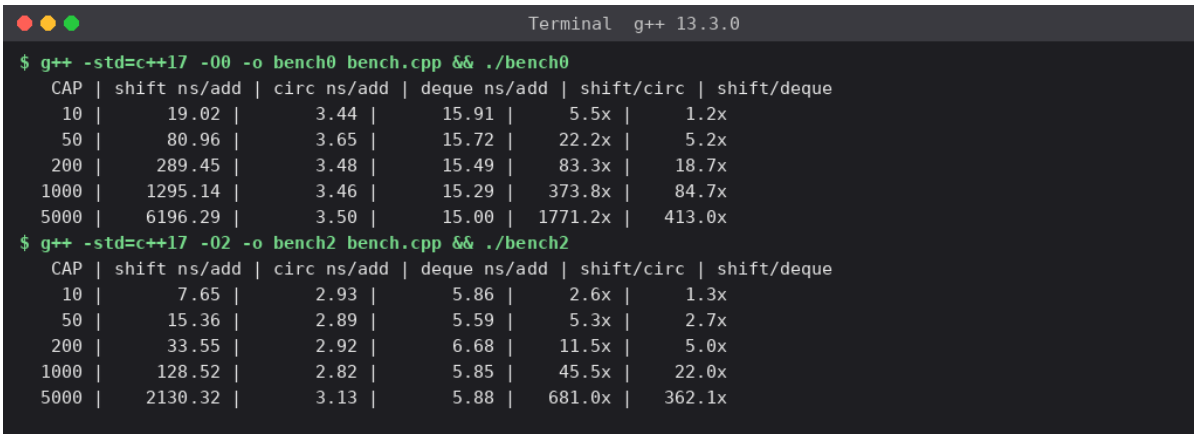


Figure 4.2 – ns per add for shifting array, circular buffer and std::deque (-O0 then -O2)

Capacity k	Shift (-O2)	Circular (-O2)	Deque (-O2)	Shift / circular
10	7.65 ns	2.93 ns	5.86 ns	2.6×
50	15.36 ns	2.89 ns	5.59 ns	5.3×
200	33.55 ns	2.92 ns	6.68 ns	11.5×
1000	128.52 ns	2.82 ns	5.85 ns	45.5×
5000*	2130 ns	3.13 ns	5.88 ns	681×

*At k = 5000 the 80 KB array no longer fits comfortably in the L1 cache, so the shift cost jumps more than linearly (cache effects); this is a measurement artefact worth noting, not a change in Big-O.

Design	Time to add	Space	Comment
Shifting array	O(k)	O(k) fixed, k·sizeof(Transaction)	Simplest; order = array order; slow when k grows (−O0: 19 → 1295 ns for k = 10 → 1000)
Circular buffer	O(1)	O(k) + one `int head`	Constant time (≈ 3 ns); needs modular indexing when reading
`std::deque`	O(1) amortised	O(k) + per-block overhead, heap allocation	Flexible and safe, but ≈ 2× slower than the hand-made circular buffer here and allocates

The measurements match the analysis: shifting grows linearly with k, the other two are flat.

Q5 – Static Members and Arrays of Objects (7 marks)

5.1 Predict the output – Token

```
#include <iostream>

class Token {
    static int next;
    int id;

public:
    Token() { id = ++next; }
    int get() const { return id; }
    static int issued() { return next - 100; }
};

int Token::next = 100;

int main()
{
    Token counter[3];
    Token extra;
    std::cout << counter[0].get() << ' ' << counter[2].get() << ' ' << extra.get()
              << ' ' << Token::issued() << '\n';
}
```

Prediction (before running): `101 103 104 4`. Reasoning: `Token::next` is initialised to 100. `Token counter[3]` runs the constructor three times, giving ids 101, 102, 103 (pre-increment: `++next` first, then store). `Token extra` is the 4th object → id 104. `counter[0].get()` = 101, `counter[2].get()` = 103, `extra.get()` = 104 and `Token::issued()` = `next - 100` = 104 – 100 = 4.

```
Terminal  g++ 13.3.0
$ g++ -std=c++17 -Wall -Wextra -o token token.cpp
$ ./token
101 103 104 4
```

Figure 5.1 – Actual output of token.cpp

Prediction vs actual: identical, no difference. **Why the counter starts at 100:** the static data member is defined once, outside the class, as `int Token::next = 100;`; it is initialised before `main` starts and exists once for the whole class, not once per object, so the objects share it. The first `++next` produces 101. The constructor ran **4 times** (3 array elements + `extra`), which is why `issued()` = 4.

5.2 Complete the code – BusTicket

TODO 1 and TODO 2 completed in `Q5/busticket.cpp` (rest unchanged):

```
explicit BusTicket(const std::string& n) : seat(0), name(n)
{
    // TODO 1: give the next seat only while seats remain
    if (issued < CAPACITY) {
        ++issued;
        seat = issued;
    }
}

static int count()
{
    // TODO 2: number of seats issued so far
    return issued;
}
```

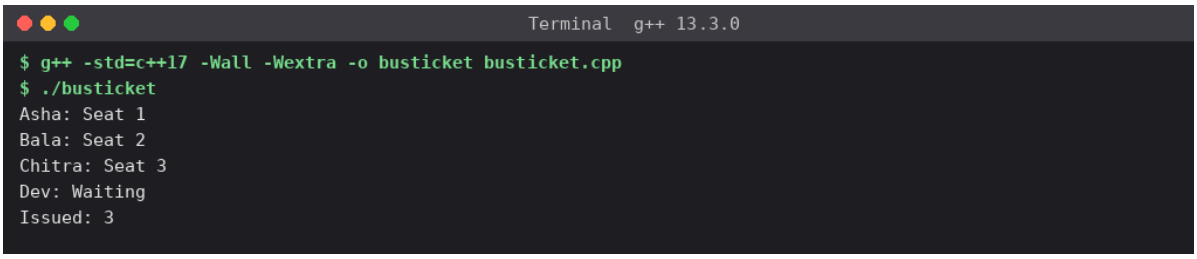


Figure 5.2 – Output equals the expected output

Expected and observed output are identical: `Asha: Seat 1`, `Bala: Seat 2`, `Chitra: Seat 3`, `Dev: Waiting`, `Issued: 3` (the fourth ticket finds `issued == CAPACITY`, so `seat` stays 0 and `print()` shows "Waiting").

5.3 Design questions

Question	Answer
Which members must be static, and where is a static data member defined?	`issued` (the count of seats given out) and `CAPACITY` (the same for every ticket) belong to the whole class → `static`. `count()` is a **static member function** because the transport office must read the count without any particular ticket. `seat` and `name` stay non-static (each ticket has its own). A static data member is only *declared* inside the class; it must be **defined exactly once outside** the class, in one .cpp file: `int BusTicket::issued = 0;` (without it the linker reports an undefined reference). `static const int` of integral type (`CAPACITY = 3`) may be initialised inside the class.
Why can a static member function not use `this` or a non-static member?	A static function is called through the class (`BusTicket::count()`), not through an object, so there is no object and no `this` pointer. Non-static members like `seat` belong to a particular object and would have no object to refer to. g++ confirms: `error: invalid use of member 'T::seat' in static member function` and `error: 'this' is unavailable for static member functions` (Q5/static_err.cpp, Figure 5.3).
How is an array of BusTicket objects created and initialised?	The only constructor needs a name, so there is no default constructor and `BusTicket trip[4];` would not compile. The array is initialised with an initialiser list `{BusTicket("Asha"), …}`; elements are constructed in index order 0…3, so the seat numbers follow the order of the list (4 constructor calls).
What happens to the count when tickets are created inside a loop or a function?	Every construction adds to the **same** static `issued` until it reaches `CAPACITY`. Objects that go out of scope are destroyed but nothing decrements the static, because it belongs to the class and lives for the whole program (shown in 5.4).

5.4 Test – more tickets than capacity; tickets created in a function called twice

`Q5/tests.cpp` creates two local tickets inside `book()` and calls it twice, then creates an array of 5 tickets. It also keeps a `created` counter to count constructions.



Figure 5.3 – Count is cumulative across calls; the array of 5 is all "Waiting"; static-function error messages

Observed: after the first call `count()` = 2 although both tickets were destroyed when `book()` returned; the second call continues from there – Chitra gets Seat 3 and Dev is Waiting – and `count()` becomes 3 with 4 constructions in total. The array of 5 created afterwards gets no seats at all (capacity exhausted). So the static count measures *seats issued over the lifetime of the program*, not the number of live objects. If "live tickets" were needed a destructor would have to decrement a separate counter.

5.5 Alternative concept and complexity analysis

Alternative: a `TicketOffice` object holding the counter and capacity, passed to every `BusTicket` (`Q5/office.cpp`), or a global variable. Output with two buses running at once: bus A issued 3 (Asha 1, Bala 2, Dev 3, Esha Waiting) and bus B issued 1 (Chitra Seat 1).

Aspect	Static member	Global variable	TicketOffice object
Time to issue a seat	O(1)	O(1)	O(1)
Space for counter	One int for the class	One int	One int per office (+ cap)
Extra space per ticket	0	0	a reference/pointer to the office, ≈ 8 bytes, if the ticket
Encapsulation	Private to the class; only class code changes it	Any code can modify it – weakest	Private inside the office; accessed through `next()`
Two buses at once	**No** – one counter for all tickets	No	**Yes** – one office per bus (demonstrated)

Conclusion: the static member is the right tool when the data truly belongs to the whole class and exactly one instance of it is meaningful; as soon as the requirement becomes "several buses", the state belongs to an object (`TicketOffice`).

Q6 – Friend Functions and Inline Functions (8 marks)

6.1 Complete the code

TODO 1 and TODO 2 completed inside the two friend functions (`Q6/time_complete.cpp`). The functions use the private `h` and `m` directly; each time is converted to minutes since midnight.

```
int minutesBetween(const Time& a, const Time& b)
{
    // TODO 1: absolute difference in minutes using private h and m
    return std::abs((a.h * 60 + a.m) - (b.h * 60 + b.m));
}

bool earlier(const Time& a, const Time& b)
{
    // TODO 2: true if a is earlier in the day than b
    return (a.h * 60 + a.m) < (b.h * 60 + b.m);
}
```

6.2 Correctness check

Hand verification: 9:45 = 585 min, 13:10 = 790 min → |585 − 790| = **205**; `earlier(arrival, departure)` asks whether 790 < 585 → **false**. Output `205 false` = expected output. Further tests (`Q6/time_tests.cpp`) include the same time twice and times on either side of noon:

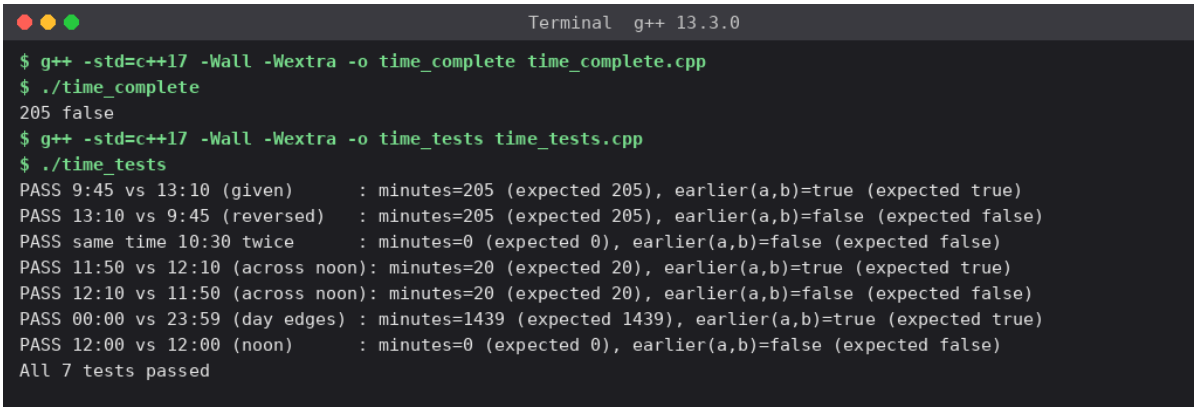


Figure 6.1 – Expected output reproduced (205 false) and 7/7 correctness tests passed

Test	Hand calculation	Result
9:45 vs 13:10	585 vs 790 → 205, earlier = true	PASS
13:10 vs 9:45	205, earlier = false	PASS
10:30 vs 10:30 (same time twice)	0 min, earlier = false (not strictly earlier)	PASS
11:50 vs 12:10 (across noon)	710 vs 730 → 20, earlier = true	PASS
12:10 vs 11:50	20, earlier = false	PASS
00:00 vs 23:59 (day edges)	0 vs 1439 → 1439, earlier = true	PASS

6.3 Design questions

Question	Answer
Why is a friend function needed when the function works on two objects equally?	The data (`h`, `m`) is private. A non-member, non-friend function cannot read it (g++: `error: 'int Time::h' is private within this context`, Q6/friend_err.cpp). A member function would work but treats one object as special (`a.minutesTo(b)`: a is `*this`, b an argument); for a symmetric operation on two objects a friend states the symmetry directly: `minutesBetween(a, b)`.
How does a friend differ from a member in how it is called and what it can access?	**Call:** a member is invoked through an object with the dot/arrow operator and receives a hidden `this`; a friend is an ordinary function called with all operands as arguments and has no `this`. **Access:** both can read private members of the class, but a friend is not in the class scope (no `Time::` qualifier, access specifiers do not apply to its declaration), is not inherited and the friendship is granted only by the class itself – and only to the named functions.
Should the small functions be inline, and what does `inline` ask the compiler to do?	`inline` is a **request** to replace the call by the function body at the call site, avoiding call/return overhead and enabling further optimisation; the compiler may ignore it (and GCC does at -O0). It also lets the definition appear in several translation units (header files) without violating the one-definition rule. These two-line functions called millions of times are ideal candidates; large functions are not, because inlining grows the code.
What would be lost if the private members were public?	**Encapsulation:** any code could set `m = 75` or `h = -3` and create an invalid time; the invariant "0 ≤ h < 24, 0 ≤ m < 60" could not be guaranteed. The representation could no longer be changed (e.g. to a single "minutes since midnight" integer) without editing every caller.

6.4 Growth investigation – 10,000,000 calls, with and without inline

`Q6/inline_bench.cpp` times 10,000,000 loop iterations, each making two calls (`minutes` and `earlier`) on pseudo-random times from a 1,024-element table (so the compiler cannot fold the result). Four designs are measured: friend functions declared `inline`; the same friend functions with `__attribute__((noinline))`; member functions (`minutesTo`, `isEarlier`); and free functions using public getters. Median of 7 runs.

```
Terminal  g++ 13.3.0
$ g++ -std=c++17 -O0 -o ib_O0 inline_bench.cpp && ./ib_O0
10,000,000 iterations (each = 2 calls); median of 7 runs, milliseconds
  friend, inline        : 58.6725 ms  (checksum 4716798628)
  friend, NOT inlined   : 58.9128 ms  (checksum 4716798628)
  member functions      : 61.5843 ms  (checksum 4716798628)
  public getters (free) : 111.726 ms  (checksum 4716798628)
$ g++ -std=c++17 -O2 -o ib_O2 inline_bench.cpp && ./ib_O2
10,000,000 iterations (each = 2 calls); median of 7 runs, milliseconds
  friend, inline        : 14.5945 ms  (checksum 4716798628)
  friend, NOT inlined   : 16.0996 ms  (checksum 4716798628)
  member functions      : 11.485 ms  (checksum 4716798628)
  public getters (free) : 11.4758 ms  (checksum 4716798628)
```

Figure 6.2 – Timings at -O0 and -O2 (identical checksum 4716798628 for all designs)

Design	-O0 (ms)	-O2 (ms)
Friend, `inline`	58.7	14.6 *
Friend, NOT inlined (`noinline`)	58.9	16.1

Design	-O0 (ms)	-O2 (ms)
Member functions	61.6	11.5
Public getters (free functions)	111.7	11.5

* Run-to-run noise at -O2 is a few milliseconds on this shared 2-core machine: a second run gave 11.5 ms (inline friend) and 16.6 ms (non-inlined friend). The ranking "inlined faster than non-inlined" was stable; the three inlined designs are within noise of each other.

Explanation. (1) At **-O0** the `inline` keyword has no effect: GCC does not inline at -O0, so inline and non-inline friends are the same (58.7 vs 58.9 ms ≈ 2.9 ns per call, dominated by unoptimised code and memory traffic). Public getters are 1.9× slower at -O0 because each getter is itself a real call (4 extra calls per comparison). (2) At **-O2** the compiler inlines the small functions and the whole loop body shrinks to a few instructions: ≈ 0.6 ns per call versus ≈ 0.8 ns when a real call is forced (about 1.1–1.45× faster). The getter-based design becomes as fast as the friend version because the getters are inlined as well. Optimisation level matters far more (−O0 → −O2 is 4–5×) than the choice friend/member/getter.

6.5 Alternative concept and complexity analysis

Design	Time per call	Measured (-O0 / -O2, 20 M calls)	Encapsulation
Friend functions	O(1)	58.7 / 14.6 ms	Private data stays private; access granted to exactly 2 named non-member functions; they must change if the representation changes
Member functions (`a.minutesTo(b)`)	O(1) – same after inlining	61.6 / 11.5 ms	Strongest: only members touch the data; asymmetric call syntax
Public getters + free functions	O(1) – same after inlining, 4 extra calls at -O0	111.7 / 11.5 ms	Weakest of the three: `h`/`m` become readable by everyone (read-only), widening the public interface

6.6 Deployment decision

> **Decision: keep `minutesBetween` and `earlier` as friend functions** (declared `inline` or defined in the header). Evidence: at -O2 friend, member and getter designs are equally fast (≈ 11.5–14.6 ms for 20 million calls), so performance gives no reason to change; at -O0 the getter design is almost twice as slow. Friends keep `h` and `m` private, add no getters to the public interface, and express the symmetric two-object operation naturally. I would switch to **member functions** if the number of functions needing the representation grows (a long friend list defeats the purpose), and to getters only if outside code genuinely needs to read hours and minutes anyway.

Evidence delivered: a justified friend-function design, correctness tests, measured timings at two optimisation levels and an evidence-based decision.

Q7 – Diagnose, Redesign and Validate (13 marks)

Program chosen: the SavingsAccount mini-statement of Q4 (shifting array of the last 10 transactions). The Q4 requirement already hinted at growth ("the bank now wants the last 50"), and Q4 produced the measurements needed for a diagnosis. The redesign was implemented as `Q7/circular.cpp` next to the unmodified `Q7/original.cpp` (both accept `-DCAP=n`).

A. Diagnose the limitation (2 marks)

Evidence from my own measurements (Q4, Figure 4.2): once the history is full, each `deposit()` costs (ns per add, -O2) 7.65 at k = 10, 15.4 at k = 50, 33.6 at k = 200, 128.5 at k = 1000; at -O0 19, 81, 289 and 1295 ns. The cost rises almost proportionally with the capacity k (≈ 1.3 ns per moved element at -O0, ≈ 0.12–0.15 ns at -O2), while the circular and deque versions stay flat at ≈ 3 ns and ≈ 6 ns.

> **Diagnosis:** the design **pins the logical start of the history to physical index 0** of a contiguous array. Discarding the oldest transaction therefore forces every surviving element to be moved one slot (`last[i-1] = last[i]`), i.e. **O(k) data movement for every transaction after the buffer is full**. The weakness is not the array itself but the coupling "oldest entry = element 0". It does not matter for k = 10 (a few nanoseconds) but becomes the dominant cost as the requirement grows to 50, 1000 or more.

B. Design objective (2 marks)

> **Objective:** make the cost of recording a transaction **independent of the history length k**, so that at -O2 and k = 1000 a `deposit()` is **at least 20× faster** than the original and stays **below 5 ns for every k from 10 to 1000**, while `miniStatement()` output remains **byte-identical** to the original for the same transactions.

C. Design of the revised solution (4 marks)

What changes: the class keeps its public interface (`deposit`, `withdraw`, `miniStatement`, `getBalance`) and the private `last[CAP]` array, but adds one private data member `head` – the index of the **oldest** entry. Logical position *i* (0 = oldest) lives at physical index `(head + i) % CAP`.

SavingsAccount (class diagram)

Private data
`long accNo`
`double balance`
`Transaction last[CAP]` – fixed array (unchanged)
`int count` – valid entries, 0…CAP
`int head` – **NEW**: index of the oldest entry

Private `void record(char t, double amt)` – **changed**: O(1) circular write

Public
`SavingsAccount(long, double)`
`double getBalance() const`
`void deposit(double)`
`bool withdraw(double)`
`void miniStatement() const` – **changed**: reads via `(head + i) % CAP`

```
record(type, amount):
    e = Transaction{type, amount, balance}
    if count < CAP:                         // still filling
        last[(head + count) % CAP] = e ;  count = count + 1
    else:                                   // full: 11th, 12th ... transaction
        last[head] = e                      // overwrite the OLDEST entry
        head = (head + 1) % CAP             // the next-oldest becomes the oldest

miniStatement():                            // newest first
    for i = count-1 down to 0:  print last[(head + i) % CAP]
```

```
// Q7/circular.cpp - the changed member functions
void SavingsAccount::record(char t, double amt)
{
    Transaction e{t, amt, balance};
    if (count < CAP) last[(head + count++) % CAP] = e;   // still filling
    else { last[head] = e; head = (head + 1) % CAP; }    // overwrite oldest: O(1)
}
// in miniStatement():  const Transaction& t = last[(head + i) % CAP];
```

C++ features used: private data members and member functions (encapsulation: no caller changes), the `%` operator for wrap-around, and a compile-time constant `CAP` (the `-DCAP=` macro only lets me benchmark several sizes). **Extra memory:** one `int` (4 bytes plus padding) per account – no extra array; total space remains O(k). **New trade-off:** the history is no longer stored in order in memory, so every read needs an index calculation with a modulo; the logic is slightly harder to read and to debug than a plain array.

Note that I did **not** just replace the array with `std::deque`: the circular-index solution removes the root cause (data movement) by changing how positions are interpreted. `std::deque` is included in the Q4 measurements only as a comparison (≈ 5.9 ns, about twice the hand-made circular buffer).

D. Predict before implementing (1 mark)

Prediction derived from the Q4 cost model (shift cost ≈ c·(k − 1) with c ≈ 0.12–0.15 ns per element at -O2, circular cost constant ≈ 3 ns):

Capacity k	Predicted speed-up of `deposit()` (-O2)	Predicted read-path (`checksum`/ statement walk)
10	≈ 2–3× faster	circular slower: ≈ 2–3× (modulo on every element)
50	≈ 5× faster	circular ≈ 2–3× slower
1000	≈ 40× faster; circular flat at ≈ 3 ns	circular ≈ 2–3× slower

Testable measure: ns per `deposit()` at k = 10, 50, 200, 1000, and ns to walk the whole history, using the same benchmark loop on both programs (`bench_add.sh`, `bench_read.sh`).

E. Implement and validate (2 marks)

Correctness first. The same scripted sequence (60 transactions mixing deposits and withdrawals) was run through both programs; the outputs are byte-identical for k = 10 and k = 50 (the `diff` printed nothing; MD5 sums of both outputs equal).



Figure 7.1 – original and circular produce identical statements at CAP = 50

Measurements (same machine, -O2, median-of-7 best ns per call, 1,000,000 deposits; reads: 2,000,000 full-history walks):

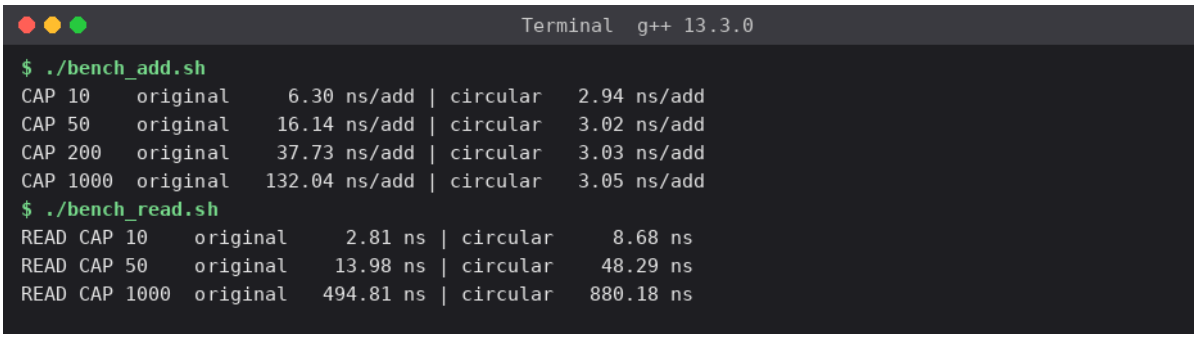


Figure 7.2 – Write path (deposit) and read path (walking the history), original vs circular

Capacity k	Original deposit	Circular deposit	Speed-up	Predicted	Original read	Circular read	Read slow-down
10	6.30 ns	2.94 ns	2.1×	2–3×	2.81 ns	8.68 ns	3.1×
50	16.14 ns	3.02 ns	5.3×	≈ 5×	13.98 ns	48.29 ns	3.5×
200	37.73 ns	3.03 ns	12.5×	–	–	–	–
1000	132.04 ns	3.05 ns	43.3×	≈ 40×	494.81 ns	880.18 ns	1.8×

The same measurements at -O0 (separate run): original 25 / 73 / 262 / 1276 ns versus circular 4.6 / 4.5 / 4.5 / 4.9 ns for k = 10 / 50 / 200 / 1000.

F. Defend the final design (2 marks)

Question	Evaluation
Was the objective achieved?	**Yes for the write path.** At k = 1000 the speed-up is 43× (target ≥ 20×); the cost is flat at 2.9–3.05 ns for all k (target < 5 ns); statements are byte-identical. The speed-up predictions held for the write path (≈ 2×, 5×, 43× vs predicted 2–3×, 5×, 40×). The read-path prediction (2–3× slower) was slightly off: 3.1× at k = 10, **3.5× at k = 50** and only 1.8× at k = 1000, where memory traffic rather than the modulo dominates.
What new costs were introduced?	(1) Reading the history is up to 3.5× slower (e.g. 14 → 48 ns at k = 50) because every element needs `(head + i) % CAP`; (2) one extra `int` per account; (3) more complex indexing code that is easier to get wrong (off-by-one at wrap-around) – which is why I verified identical output before measuring; (4) the data is not in chronological memory order, so e.g. a block copy of the history needs two segments. The read cost could be reduced by walking the two contiguous segments instead of using `%` per element.
Where may the original be preferable?	For the real ATM requirement (k = 10) the difference is only 3 ns per transaction, while printing a mini-statement through `std::cout` takes microseconds – the I/O dwarfs both designs. If the statement is read much more often than transactions are written, or the code must be maintained by beginners, the simple shifting array (faster reads, trivial indexing) is the better engineering choice. The redesign pays off when k grows (50 and above) and writes are frequent.
Conclusion	The redesign correctly removes the diagnosed cause (O(k) element movement) and meets its measured objective, at the price of slower reads and more complex indexing. It is **not unconditionally better**: I would choose the circular buffer for capacities ≥ 50 or write-heavy workloads, and keep the plain shifting array for the 10-entry mini-statement where simplicity wins.

Appendix A – README.md (as submitted in the repository)

```
# DSA01-192565075-Assignment2

| | |
|---|---|
| **Name** | `<YOUR NAME>` |
| **Register number** | 192565075 |
| **Course** | Data Structures and Algorithms (DSA01) — Assignment 2 (CO2) |
| **Compiler** | g++ 13.3.0 (Ubuntu 13.3.0-6ubuntu2~24.04.1) on Linux x86-64 |

## Compile command

```
g++ -std=c++17 -Wall -Wextra -o <name> <name>.cpp
```
Timing programs add `-O0` or `-O2`; Q7 programs add `-DCAP=<n>`; Q2 sanitizer runs add `-g
-fsanitize=address`.

## Layout and how to run (one folder per task; output screenshots in `QN/screenshots/`)

| Task | Files | Run | Result |
|---|---|---|---|
| Q1 | `predict.cpp`, `complete.cpp`, `three_ways.cpp` | `g++ ... -o x x.cpp && ./x` |
Prediction `1 12 13 / 1 100 / 16 13 6` matched. Complete-the-code prints 45000 then 40500.
Value version leaves the fee unchanged; reference and pointer change it. |
| Q2 | `marks.cpp`, `vector_grow.cpp`, `alternatives.cpp` | `./marks` ; `./marks bad`
(dangling call) | Reference return lets `f() += 2`. Returning a local reference gives `-
Wreturn-local-addr` and a segfault (exit 139). Vector growth moves the element (ASan: heap-
use-after-free). |
| Q3 | `ambiguous.cpp` (fails to compile), `ambiguous2.cpp` (fails), `charge.cpp`,
`alt_parcel.cpp` | `./charge` | Six predicted overload selections all matched: 30.04, 70.00,
80.00, 120.00, 115.00, 90.00. |
| Q4 | `account.cpp`, `bench.cpp` | `./account` ; `./bench2` | 11th+ transactions discard
the oldest. Shifting cost grows with capacity; circular index is constant. |
| Q5 | `token.cpp`, `busticket.cpp`, `tests.cpp`, `static_err.cpp` (fails), `office.cpp` |
`./busticket` | Token prints `101 103 104 4`. BusTicket matches expected output (Seat 1-3,
Dev waiting, Issued 3). |
| Q6 | `time_complete.cpp`, `time_tests.cpp`, `inline_bench.cpp`, `friend_err.cpp` (fails) |
`./time_complete` | `205 false`; 7/7 tests pass; inline timings at -O0 and -O2 recorded. |
| Q7 | `original.cpp`, `circular.cpp`, `bench_add.sh`, `bench_read.sh` | `./bench_add.sh` |
Circular buffer: ~3 ns per deposit for every capacity; original grows linearly (6 ns at 10
to 132 ns at 1000). Output identical. Reads are slower in the redesign. |

Files marked "(fails)" are intentional compile-error demonstrations; the error text is in
the report.
Timings vary by machine and load (about +-10 %); medians of 7 runs are reported.
```

Appendix B – GitHub submission checklist and commit commands

The assignment requires a repository named `DSA01-<RegisterNumber>-Assignment2`, shared with the faculty, one folder per task, and **at least one commit per task** (a single upload on the last day is not accepted). The source folder supplied with this report is laid out exactly that way. Suggested commands (run on your own machine after executing every program yourself):

```
git init DSA01-192565075-Assignment2 && cd DSA01-192565075-Assignment2
# copy README.md and the Q1..Q7 folders in, one task at a time:
git add README.md Q1 && git commit -m "Q1: call by value/reference/pointer"
git add Q2 && git commit -m "Q2: return by reference and dangling references"
git add Q3 && git commit -m "Q3: default arguments and overloading"
git add Q4 && git commit -m "Q4: SavingsAccount class with array"
git add Q5 && git commit -m "Q5: static members and arrays of objects"
git add Q6 && git commit -m "Q6: friend and inline functions"
git add Q7 && git commit -m "Q7: diagnose, redesign and validate"
```

```
git branch -M main
git remote add origin https://github.com/<your-username>/DSA01-192565075-Assignment2.git
git push -u origin main
```

Then add the faculty as a collaborator (or make the repository public) and paste the link on the cover page.

Appendix C – Summary of complexity results

Task	Alternative compared	Time	Space	Own measurement
Q1	In-place vs return new fee; by value/reference/ pointer/const&	O(1) all	8 bytes copied (by value / pointer) vs 0 (reference); O(s) for large objects by value	Outputs 45000 / 40500 / 36450 / 40500
Q2	Reference vs index vs pointer	O(n) all	O(1)	1.74–1.89 ns/element for n = 10^3…10^7
Q3	Overloads+defaults vs Parcel struct/template	Compile-time resolution, O(1) run time	≈16 bytes vs struct fields	All six predicted selections correct
Q4	Shift vs circular vs deque	O(k) vs O(1) vs O(1)	O(k) each (+1 int / + block overhead)	-O2 k = 1000: 128.5 / 2.8 / 5.9 ns
Q5	Static vs global vs TicketOffice	O(1) all	1 int (+ pointer per ticket for office)	Count 2 → 3 over two calls; 4 constructions
Q6	Friend vs member vs getters; inline vs not	O(1) all	O(1)	-O2: 11.5–14.6 ms inline vs 16.1 ms not (20 M calls)
Q7	Original vs circular (Q4 program)	O(k) vs O(1) per add; reads O(k) both	O(k) + one int	k = 1000: 132 → 3.05 ns (43×); reads 1.8–3.5× slower